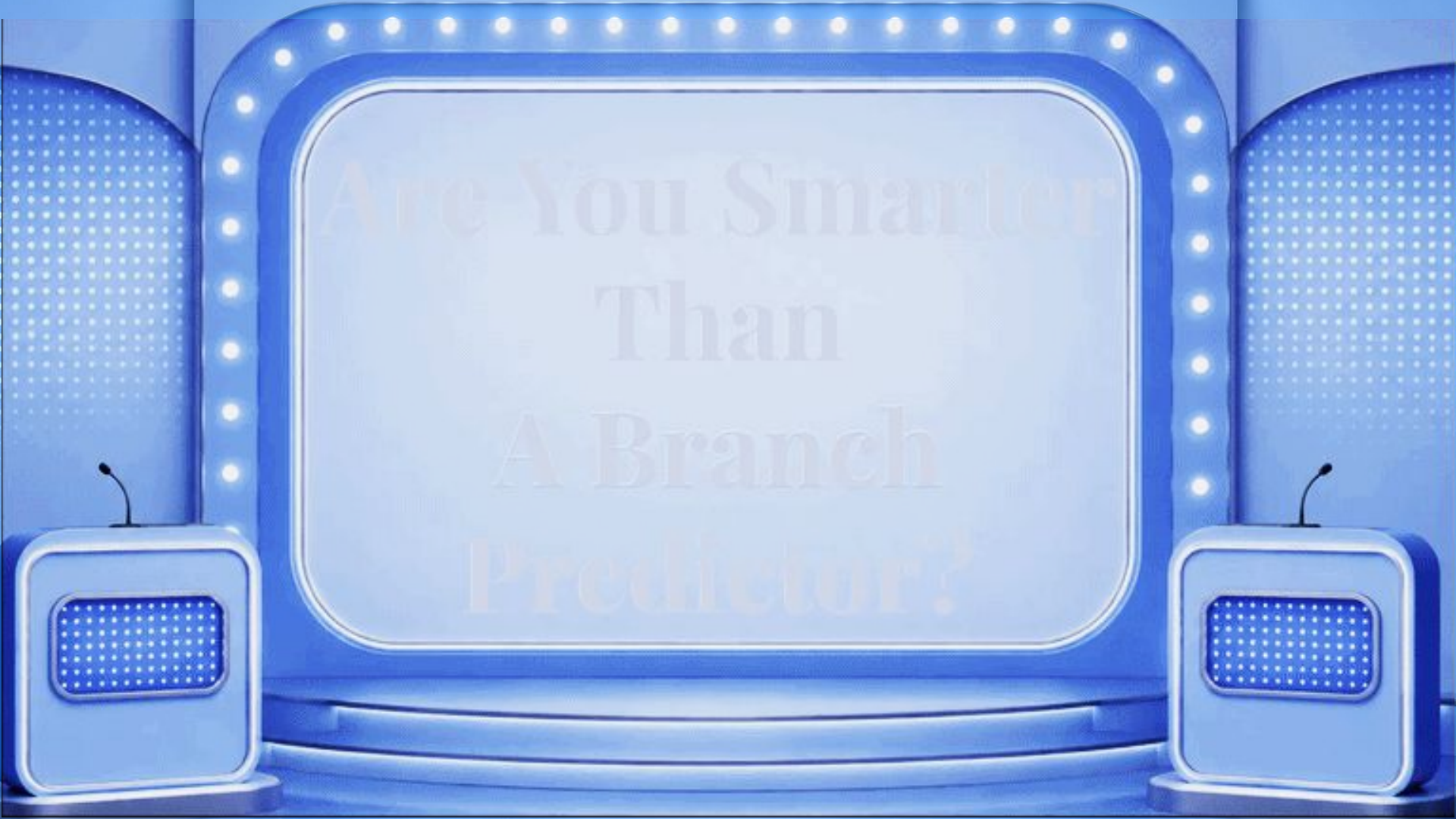


Are You Smarter Than A Branch Predictor?

MICHELLE FAE D'SOUZA

The image shows a stylized blue game show set. In the center is a large, rounded rectangular screen with a white border, surrounded by a ring of small white lights. The screen displays the text 'Are You Smarter Than A Branch Predictor?' in a light blue, serif font. On either side of the screen are blue podiums with a grid of small lights on the front and a microphone on top. The background consists of blue panels with a grid of small lights.

Are You Smarter
Than
A Branch
Predictor?

My Name is Michelle



Michelle D'Souza, Software Engineer @ Bloomberg

switch

cond ? res1 : res2

for

if

for

if

while

Branches Are Everywhere

while

if

for

if

while

switch

cond ? res1 : res2

Understanding them makes you a more informed programmer.

Source: flamingtext.com

Image generated by ChatGPT using DALL·E (OpenAI) based on user-provided description.

Today's Agenda



The Pipeline

Brief Outline of the Branch Prediction Pipeline

The Cutest Ever



Novelty Erasers!



Each one is unique



The Compiler's Game

Identifying Branches (We may not get here :))

⚠ ERASER SAFETY NOTICE

A lot of these are microbenchmarks, done on my "hybrid Intel Raptor Lake mobile CPU" (x86)

Soft erasers. Questionable aim.

Enter Michelle's line of fire at your own risk.

The game show accepts no liability for eraser-related damages. :)





```
if (x > 0) {  
    foo();  
} else {  
    bar();  
}
```



```
test    edi, edi        ; x > 0 ?  
jle     .else           ; branch if x <= 0  
  
call    foo  
jmp     .done  
  
.else:  
    call bar  
  
.done:
```



```
85 FF 7E 05 E8 FA E3 76 10 81 B1 87 8C ..
```

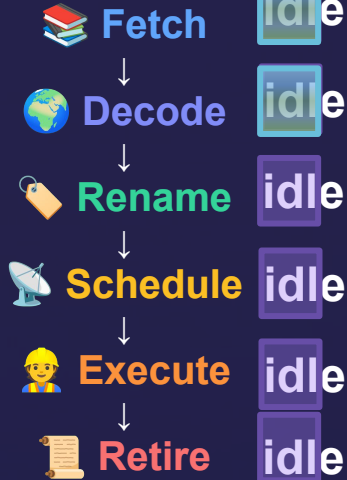
The Pipeline

// if (x > 0)

With NO branch
prediction
hardware



```
if (x > 0) {  
    foo();  
} else {  
    bar();  
}
```



The branch
was taken!

*Modern CPUs fetch multiple instructions at a time, as bytes rather than as Assembly / C++

Source: giphy (<https://i.pinimg.com/originals/a1/3f/68/a13f687f15fbbd72be84bb08f5203bc571>)

Image generated by ChatGPT using DALL·E (OpenAI) based on user-provided description.

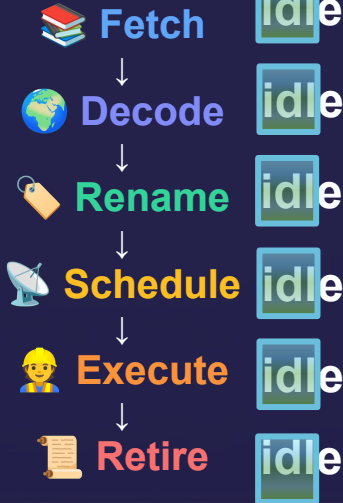
The Pipeline

// if (x > 0)

With NO branch
prediction
hardware



```
if (x > 0) {  
    foo();  
} else {  
    bar();  
}
```



The branch
was taken!

At fetch, the CPU consults its branch-prediction structures to determine where it should fetch from next.

The Pipeline

// if (x > 0)

With branch prediction hardware

branch
C.

Just bytes

```
if (x > 0) {  
    foo();  
}
```

Branch Target Buffer

If the branch is taken, here is where it will go

Tag / Fetch Address	Target Information
0x1007	JNE → target 0x2000

Modern processors typically lose 10 to 20+ clock cycles due to flushing the pipeline and restarting execution on the correct path.

And that's just the misprediction recovery. If the correct path then encounters an instruction or data cache miss, you can pay additional memory latency on top of that.

(20 cycles ≈ 9 ns @ 2.2 GHz . Tiny to us, expensive to a CPU, especially if it repeats)

The branch was NOT taken!

(fall-through)

Source

PART 1 (WARM UP)!

Decoding the Hardware: **How Predictors Think**

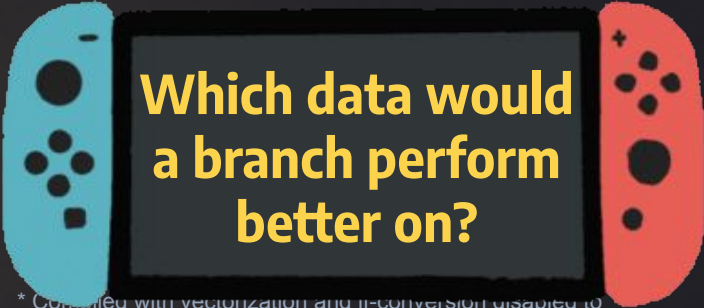
Note: Assume all optimizations are turned **ON**



It's all in the pattern

Same branch taken rate. Very different predictability.

Patterned data: TTTTF TTTTF TTTTF...



**Which data would
a branch perform
better on?**

* Compiled with vectorization and if-conversion disabled to preserve the real conditional branch.

Predictors don't just "guess the percentage"; they are pattern-matching machines that track history.

<https://godbolt.org/z/qvMejM48T>



It's all in the pattern

Predictable

PART 2

The Optimization Toolkit: Tactics for Speed

Note: Assume all optimizations are turned **ON**

One line of code ...

Current speed:

2352 ms

```
int process(const std::vector<int>& prices) {  
    int sum = 0;  
  
    for (int price : prices) {  
        if (price > 0) // branch  
            sum += costly(price);  
    }  
  
    return sum;  
}  
  
for (int analysis = 0; analysis < 100; ++analysis)  
    result += process(prices);
```

Len of prices == 5 Million

<https://godbolt.org/z/8TzGrEe7d>

Adding one line of code can make this much faster. What is it?

2352 ms

But it's not just adding a line: it also requires removing or changing others. It could work well in some cases, though.



```
int process(const std::vector<int>& prices) {  
    int sum = 0;
```

```
    for (int price : prices) {
```

```
        sum += (x > 0) * costly(x);
```

```
    }
```

```
    return sum;
```

```
}
```

```
for (int analysis = 0; analysis < 100; ++analysis)  
    result += process(prices);
```

Branchless!

435 ms

2352 ms



```
int process(const std::vector<int>& prices) {  
    int sum = 0;  
  
    for (int price : prices) {  
        if (price > 0)           // branch  
            sum += costly(price);  
    }  
  
    return sum;  
}
```

Branchless!

435 ms

```
std::sort(prices.begin(), prices.end());
```

Sorting!

495 ms

```
for (int analysis = 0; analysis < 100; ++analysis)  
    result += process(prices);
```

2352 ms



```
int process(const std::vector<int>& prices) {  
    int sum = 0;  
  
    for (int price : prices) {  
        if (price > 0)           // branch  
            sum += costly(price);  
    }  
}
```

Partitioning the values so that all ≤ 0 values come first,
followed by all > 0 values, also works.

```
std::partition(prices.begin(), prices.end(), [](int x) { return x <= 0; });
```

```
for (int analysis = 0; analysis < 100; ++analysis)  
    result += process(prices);
```

Branchless!

435 ms

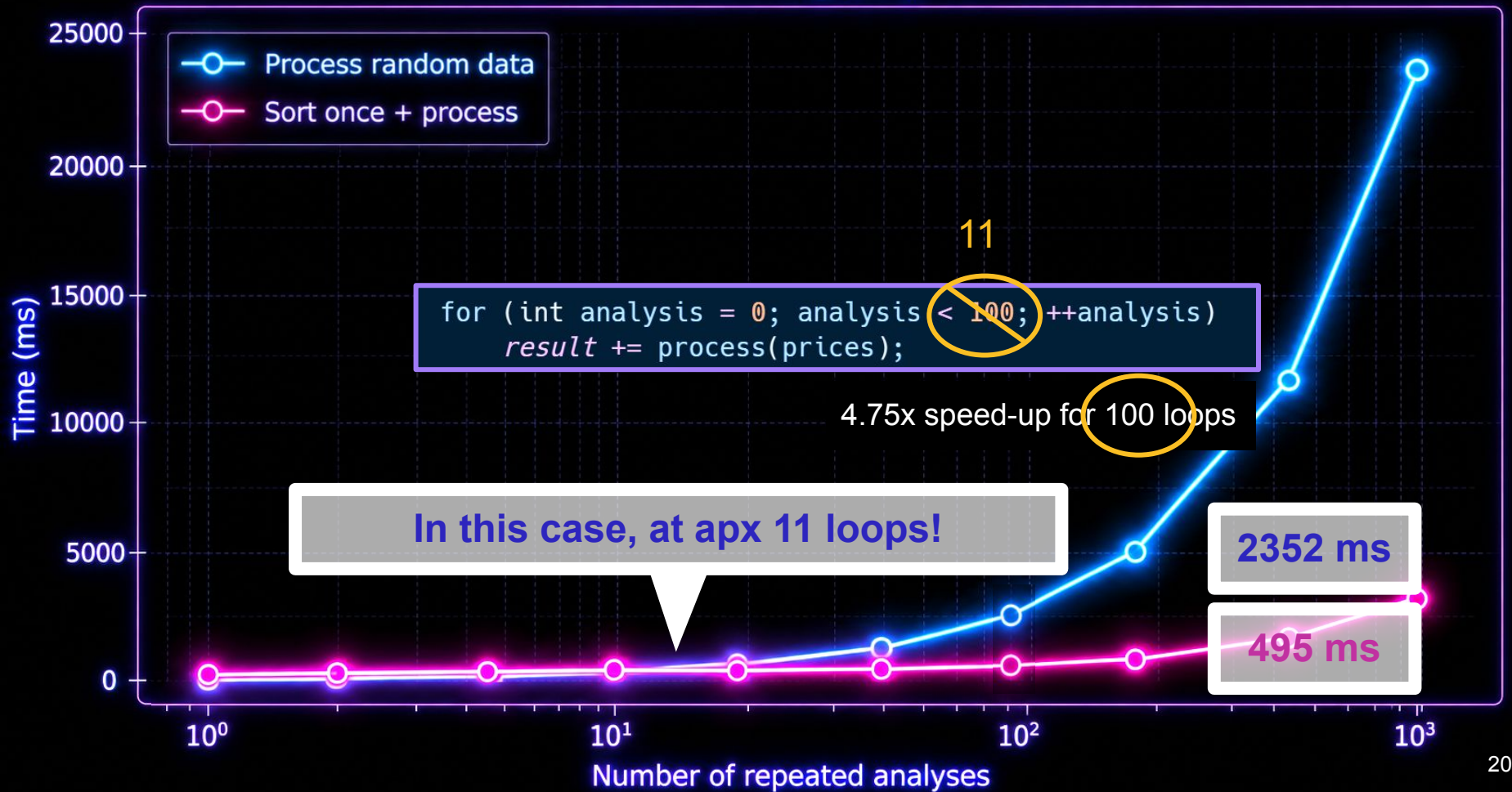
Partition!

300 ms

495 ms



When does sorting pay for itself?



Different speed up approaches

① Up to ~11 analyses:



② ~11 to ~21 analyses:



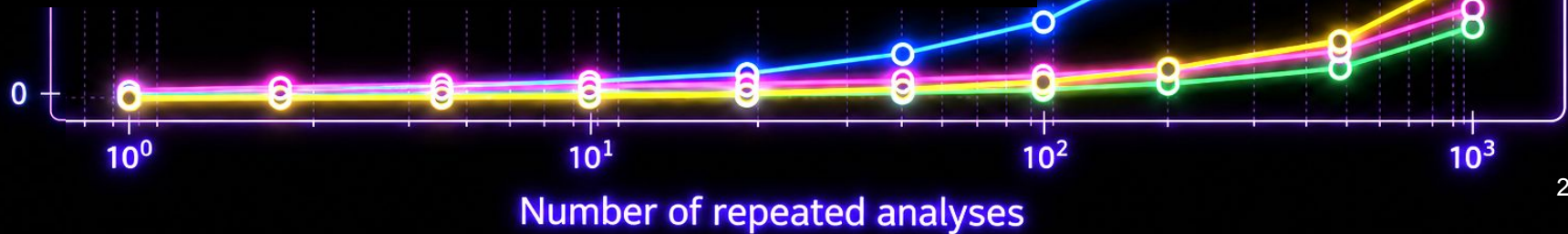
③ ~21 to ~189 analyses:



④ ~189 analyses onward:



"<" means faster than



Honey, I Shrunk the Dataset!

What if instead of looping through **5M** elements in prices, it was just **20**?

```
for (int analysis = 0; analysis < 100; ++analysis)  
    result += process(prices);
```

① 1 – 2 analyses: Branchless multiply < Partition < Normal/random < Sort

Differences are tiny: branchless leads by just 0.000121 at 1 analysis and 0.000021 at 2.

② ~5 – 50 analyses: Partition < Branchless multiply < Sort (others vary) < Normal/random

At this scale, the differences are tiny and the exact ordering isn't stable.

③ 50+ analyses: Partition < Sort < Normal/random < Branchless multiply

The one-time cost of reorganizing the data now pays off.

Similar to the earlier

① Up to ~11 analyses: 5 mill case

Similar to the earlier

④ ~189 analyses onward: 5 mill case, but

Branchless multiply < Normal/random

Why did **branchy normal / random** do better than **branchless multiply** in the 20 element case (here) but not in the 5 million element case?



How Does The Branch Predictor Work?

Direction Prediction

"Taken or Not Taken?"

History-Based Predictors

- Local history predictors
- Global history predictors
 - TAGE
 - Perceptron / Neural
 - gshare / gselect

Specialized Predictors

- Loop predictor

Target Prediction

"Where do we go?"

Predictor Types

- BTB (Branch Target Buffer)
- Indirect-target predictor
- Return Address Stack

Prediction:
TAKEN

Target:
0x500



Local History

Local History (The Same Branch)

What that exact branch did last time

last x bits of branch PC

↓
Local history table

index	history
0	101101
1	001011

Index 36, Pattern: 101101

↓
Shared pattern history table

↓
Prediction: STRONGLY TAKEN

pattern index

2-bit saturating counter

000000

Strongly Not Taken (00)

101101

Strongly Taken (11)

111111

Weakly Taken (01)

What is the “**Strongly**” and “**Weakly**” next to taken/not taken?

Local History and Global History

Local History (The Same Branch)

What that exact branch did last time

Global History (Different Branches)

Depending on recent other branches...

The value of x:
Modern performance-critical silicon uses
100 to 500+ bits

Global History Predictors, Conceptually...

if (x > 0)

Every **branch instruction** has an **address**

0x4e1234

The predictor asks...

If yes, it looks up some history.

CPU remembers the last x branches across the whole program

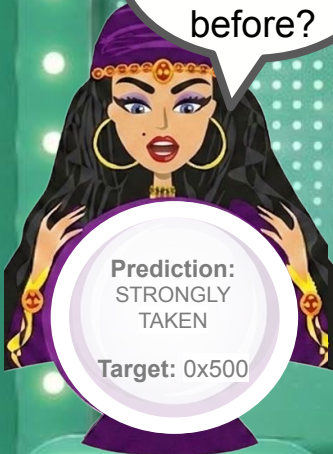
Global History Register (GHR)

1 1 0 0 0 1 0

Hash(,)

0x83472	11 (Strongly taken)
0x0221a	01 (Weakly taken)

Have I
seen this
branch
before?



TAGE

(Tagged GEometric history length predictor)

Different branches require different amounts of history.
So remember several history lengths simultaneously.

address
0x4e1234



In the table

Table with
History Len 4

0x42ef

111101101110110010010



NOT in the table

Table with
History Len 8

0xab31



In the table

Table with
History Len 16

0x0318



In the table

Table with
History Len 32

0x0113

Strongly not taken, Weakly not taken,
Weakly taken, Strongly taken
Hash(PC , history)



Perfect!

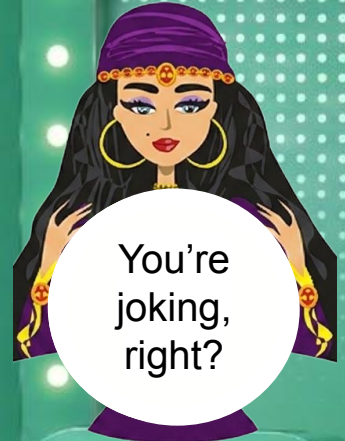
BTW, computing the hashes and looking them up in the tables happen in parallel,
so it's faster than you're being shown. There are also faster alternatives to hashes.

We've covered **TAGE**

- + How Global History and Local History Predictors work



But wait,
there's more!



Perceptron

Perceptron Table
With N perceptrons

Perceptron #632
[w0, w1, w2, ... wX]

E.g. address mod N

address

0x4e1234

Then the global history is applied to **those weights**

Global History Register (GHR)

Taken 1 0 0 1 0 0 1 0

Mathematically: Not Taken -1 -1 1 -1 -1 1 -1

$$w_0 + (1 * w_1) + (-1 * w_2) + (-1 * w_3) + (1 * w_4) + (-1 * w_5) \dots$$

Learned dynamically from previous branch behavior

Prediction:
TAKEN

Target:
0x500

Are the weights
inside the
perceptron
hardcoded when
the program
starts?

Loop Predictors!

Some branches tend to repeat a certain number of times!

extract/mix PC bits

address
0x4e1234

Tag (Branch Identity)	Expected Trip Count	Current Iteration Count	Exit Direction	Confidence
0x021	7	3	T	Low (01)
0x4e1	9	0	F	Medium (10)

TTTTTTTTTT F
TTTTTTTTTT F
TTTTTTTTTT F
TTTTTTTTTT F

Confidence goes up

Prediction:
TAKEN

Target:
0x500

Hybrid Predictors

Modern CPU

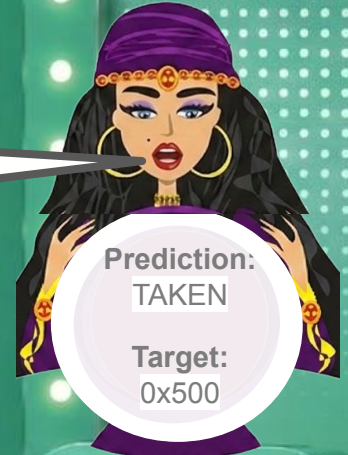
predictors

Example
score = v

$B + \dots + \text{bias}$

Ok... but what
about the question
earlier??

+ == Taken
- == Not Taken



Prediction:
TAKEN

Target:
0x500

Branchy Random Case

```
int process(const std::vector<int>& prices) {  
    int sum = 0;  
  
    for (int price : prices) {  
        if (price > 0) // branch  
            sum += costly(price);  
    }  
  
    return sum;  
}  
  
for (int analysis = 0; analysis < 100; ++analysis)  
    result += process(prices);
```

Pattern!!!

Random → T F T T T F T ... (20 outcomes)

Random → T F T T T F T ... (20 outcomes)

Random → T F T T T F T ... (20 outcomes)

...

Why did **branchy random**
do better than **branchless multiply**
in the 20 element case but not
in the 5 million element case?

Len prices
== 20

Finite history length

The predictor components remembers only the last x outcomes.

194

Taken Branches

On Raptor Cove P-core (reverse engineered):

A 388-bit Path History Register represents path information from up to **194 taken branches**.

Our earlier example repeated the pattern after **5 million elements**.

**Finite predictor component storage be the reason for
branchy random processing not doing so well!**

Branches are best ...

... On predictable data!

And depending on your program, branchless *might* be faster

3 more question, but
not enough time?

Where you place your branch ...

Which code would perform better?

```
for (auto x : data) {  
    if (debug_enabled)  
        log(x);  
    sum += work(x);  
}
```

1

GCC 16.1

```
if (debug_enabled) {  
    for (auto x : data) {  
        log(x);  
        sum += work(x);  
    }  
} else {  
    for (auto x : data) {  
        sum += work(x);  
    }  
}
```

2

Data is `std::vector<int>`

<https://godbolt.org/z/qjTbqMa66>

Compilers can do loop unswitching!

So the branch is **not inside the hot loop** anymore.

```
for (auto x : data) {  
    if (debug_enabled)  
        log(x);  
  
    sum += work(x);  
}
```

1

GCC 16.1

Clang 22.1.0

MSVC 19.51(/O2)

```
if (debug_enabled) {  
    for (auto x : data) {  
        log(x);  
        sum += work(x);  
    }  
} else {  
    for (auto x : data) {  
        sum += work(x);  
    }  
}
```

2

<https://godbolt.org/z/qjTbqMa66>

But also - which code would you write?



```
for (auto x : data) {  
    if (debug_enabled)  
        log(x);  
  
    sum += work(x);  
}
```

1

No code duplication!
WAY better - especially if no / minute performance add



```
if (debug_enabled) {  
    for (auto x : data) {  
        log(x);  
        sum += work(x);  
    }  
} else {  
    for (auto x : data) {  
        sum += work(x);  
    }  
}
```

2

But also - which code would you write?

```
for (auto x : data) {  
    if (debug_enabled)  
        log(x);  
  
    sum += work(x);  
}
```

1

```
if (debug_enabled) {  
    for (auto x : data) {  
        log(x);  
    }  
}  
  
for (auto x : data) {  
    sum += work(x);  
}
```

3

2 for loops in example 3 means evicting + refetching data into your cache.

When 20 million elements in data,

Example 1 is 1.3 times faster

Median one pass: 41.978 ms

Median two passes: 54.597 ms

<https://godbolt.org/z/nTs7aq94j>

Where you place your branch ...

... could be optimized in the generated
Assembly code ...

2 more questions,
but not enough time?



BRANCHES

WORK BETTER!

Summary



1)

On predictable patterns ➡ SORTING HELPS

2)

Loop unswitching can be done automatically by the compiler in some cases.





WE'RE SKIPPING THE LAST ROUND!

No time to hunt down the branches today...

The Compiler's Game:

Want the missing round? Catch it next week at
NDC TechTown or in the corresponding recording.

Note: Assume all optimizations are turned ON

But there's another way to hunt down
problematic branches

How Do You Spot These Sneaky Branches?

Rely on measurements, not hints; profile and verify before you optimize

Method

Profile

Profile
verify the
mispre

Also if you have two lines of code that map to the same branch predictor hardware, this can cause interference (their histories are merged)!

Be frugal with branches!

- Eduardo Madrid

Prof · Linux perf record /

misprediction bottlenecks

er generated (Assembly

for a change. Quantify *why* it
(C), branch misses (% and

asure the actual end-to-end

speedup and run enough iterations to get stable results.



Intel VTune · AMD uProf · Linux perf

Celero · Google Benchmark

How Do You Spot These Sneaky Branches?

Rely on measurements, not hints; profile and verify before you optimize

Other tips

Profile Guided Optimization (PGO).

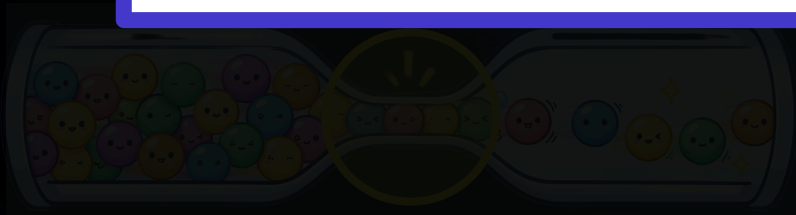
Benchmark with data mimicking production patterns.

01/ IDENTIFY BOTTLENECKS Intel VTune · AMD uProf · Linux perf record / report: Find performance hotspots and diagnose branch misprediction bottlenecks using hardware performance counters.

02/ CHECK ASSEMBLY: Understand what your compiler generated (Assembly structure).

Use PGO so the compiler sees real-world branching patterns during training runs, enabling it to generate better branch layouts!!!

- **Example with GCC:** `g++ -fprofile-generate app.cpp ./app # Run under normal workload`
- `g++ -fprofile-use app.cpp # Recompile using the generated profile data`



Intel VTune · AMD uProf · Linux perf
Celero · Google Benchmark

Bloomberg

Engineering

Before We Wrap Up ❤️

Interested in C++ @ Bloomberg?

We are hiring: bloomberg.com/engineering

Career Opportunities



Learn more about our work



TechAtBloomberg.com

Thank You!

For tuning into today's episode!

Interested in C++ @ Bloomberg?

We are hiring: [bloomberg.com/engineering](https://www.bloomberg.com/engineering)

Career Opportunities



Learn more about our work



TechAtBloomberg.com

© 2026 Bloomberg Finance L.P. All rights reserved.

Special thanks to Dietmar Kuhl, Jess Winer, Eduardo Madrid, Chris Cotter, Alex Lo, Marc Kurtz, Clark Yang, Tom Hulton-Harrop, Chaim Haas, Sherry Sontag, and my mum

Engineering

Bloomberg

P.S. If there is time and there are no more questions, we can do a bit of the [third section](#)!

P.P.S. Be the first to guess the game show that this talk was named after, and win a prize :)